Universidad Zaragoza

Automatic Machine Learning

Applications of Deep Learning (69158)

Rubén Martínez Cantín
Dpto. Informática e Ingeniería de Sistemas
Universidad de Zaragoza

Machine Learning

"Machine learning is the science of getting computers to act without being explicitly programmed."

by Andrew Ng

Zoubin Ghahramani said that he often heard that:
"I'd like to use machine learning, but I can't invest much time."

Goal of AutoML

AutoML

The goal of AutoML is to automate all parts of machine learning
(as needed) to *support* users efficiently building their ML-applications.

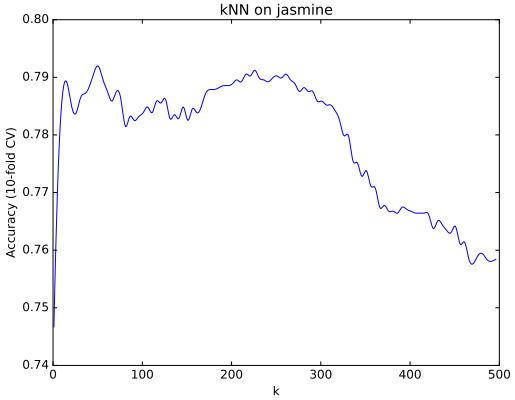
Informal Definition: AutoML System

Given

- a dataset
- a task (e.g., regression or classification)
- a cost metric (e.g., accuracy or RMSE)

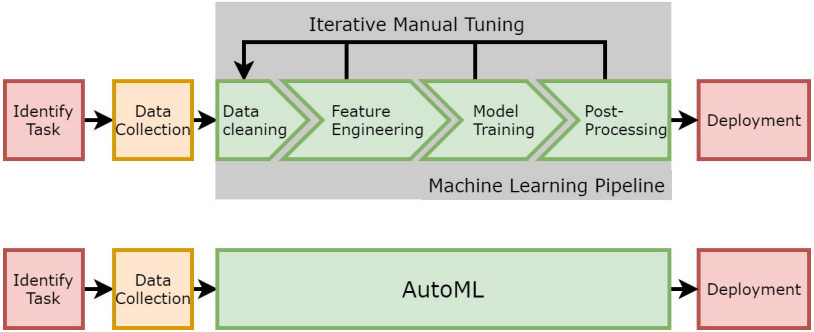
an AutoML system automatically determines the approach that performs best for this particular application.

A Simple Example with k-NN



- k-nearest neighbors is one of the simplest ML algorithms
- Size of neighbourhood (k) is very important for its performance
- The performance function depending on k is quite complex (not at all convex)

ML vs AutoML



With AutoML, we ...

- support ML users
- improve the efficiency of developing new ML applications
- reduce the required ML-expertise
- might achieve better performance than developers w/o AutoML

Challenges in AutoML

1. Each dataset potentially requires different optimal ML-designs
   - Design decisions have to be made for each dataset again
2. Training of a single ML model can be quite expensive (e.g., hours, days or weeks)
   - often, we cannot try many design decisions
3. the mathematical relation between design decisions and performance is (often) unknown
   - gradient-based optimization is not directly possible
4. optimization in highly complex spaces
   - incl. categorical choices, continuous parameters, conditional dependencies

Building a Successful AutoML Tool

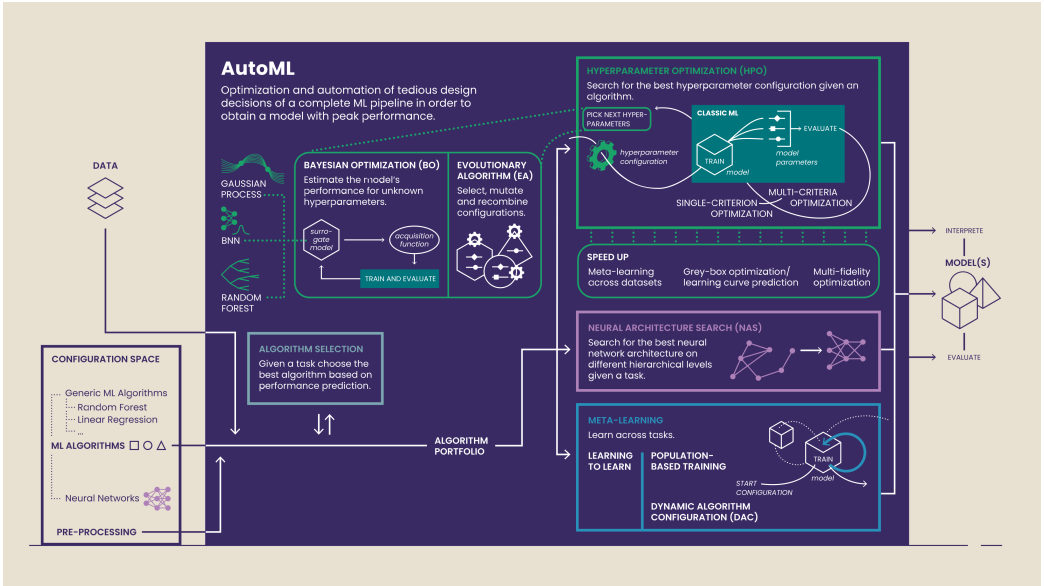
$\rightsquigarrow$ There is not a single way for building successful AutoML tools, but there are well established ideas.

- **HPO** Search for the best hyperparameter configuration of a ML algorithm
- **CASH** Search for the best combination of algorithm and hyperparameter configuration
- **NAS** Search for the architecture of neural network

- **Selection** Predict the best algorithm (and its hyperparameter configuration)
- **DAC** Predict the best hyperparameter configuration for an algorithm state at a given time point

$\rightsquigarrow$ But how can we combine all of these?

AutoML overview



Model Parameters vs. Hyperparameters

Model parameters are optimized during training, typically via loss minimization. They are an **output** of the training.

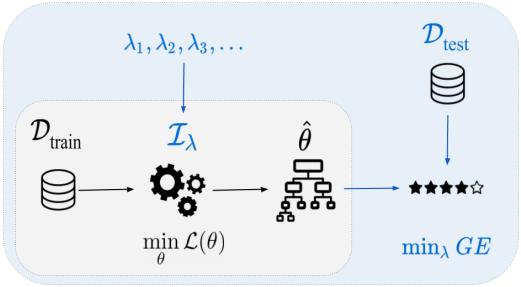
- Example: Network weights and biases.

Hyperparameters (HPs) must be specified *before* the training, they are an **input** of the training. Hyperparameters often control the complexity of a model or any computational part of the training process.

- Examples: Number of layers, learning rate, etc.

Hyperparameter Tuning: A bi-level optimization problem I

We face a **bi-level** optimization problem:



The inner loop optimizes the model parameters θ using the loss function $\mathcal{L}$ (e.g.: MSE, cross-entropy, etc.). The outer loop optimizes the hyperparameters λ for the performance metric (e.g.: generalization error (GE)).

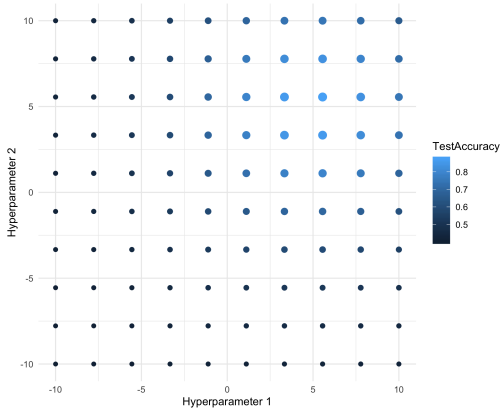
Hyperparameter Tuning: A bi-level optimization problem II

The components of a tuning problem are:

- The dataset (warning: at least two level splits *train-validation-test*)
- The learner (possibly: several competing learners?) that is tuned
- The learner's hyperparameters and their respective regions-of-interest over which we optimize
- The performance measure, as determined by the application.
  Not necessarily identical to the loss function that defines the loss minimization problem for the learner!

Grid search I

- Simple technique which is still quite popular, tries all HP combinations on a multi-dimensional discretized grid
- For each hyperparameter a finite set of candidates is predefined
- Then, we simply search all possible combinations in arbitrary order



Grid search over 10x10 points

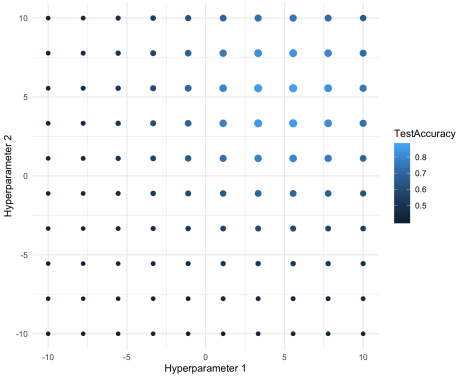
Grid search II

Advantages

- Very easy to implement
- All parameter types possible
- Parallelizing computation is trivial

Disadvantages

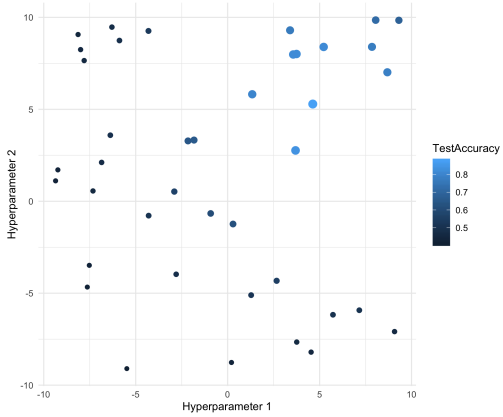
- Scales badly: Combinatorial explosion
- Inefficient: Searches large irrelevant areas
- Low resolution in each dimension
- Arbitrary: Which values / discretization?



Grid search over 10x10 points

Random search I

- Small variation of grid search
- Uniformly sample from the region-of-interest



Random search over 100 points

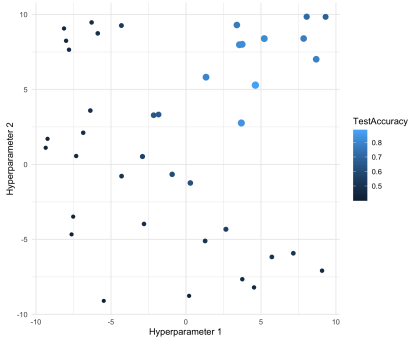
Random search II

Advantages

- Very easy to implement, all parameter types possible, trivial parallelization
- Anytime algorithm: more budget $\Rightarrow$ better performance.
- No discretization required

Disadvantages

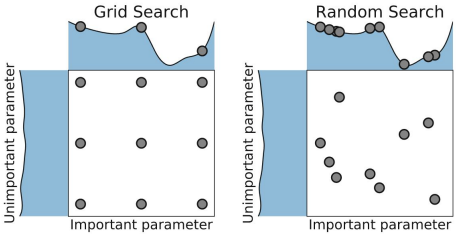
- Inefficient: many evaluations in areas with low likelihood for improvement
- Scales badly: high dimensional hyperparameter spaces need *lots* of samples to cover.



Random search over 100 points

Grid search vs. Random search

- With a tuning budget of T only $T^{\frac{1}{d}}$ unique hyperparmeter values are explored for each $\lambda_1, \ldots, \lambda_d$ in a grid search.
- Random search will (most likely) see T different values for each hyperparameter.
- Grid search can be disadvantageous if some hyperparameters have little or no influence on the model.



Comparison of grid search and random search. [Hutter et al. 2019]

Actually...


SIGOPT
GRID SEARCH
WWW.NOGRIDSEARCH.COM
SIGOPT
GRID SEARCH
WWW.NOGRIDSEARCH.COM
SIGOPT
RANDOM SEARCH
WWW.NORANDOMSEARCH.COM
SIGOPT
GRID SEARCH
WWW.NOGRIDSEARCH.COM

Global Blackbox Optimization

- Consider the global optimization problem of finding:

$$\boldsymbol{\lambda}^* \in \arg\min_{\boldsymbol{\lambda} \in \boldsymbol{\Lambda}} f(\boldsymbol{\lambda})$$

- In the most general form, function f is a blackbox function:

  - Only mode of interaction with f: querying f's value at a given $\boldsymbol{\lambda}$
  - Function f may not be available in closed form, not convex, not differentiable, noisy, etc.

- Hyperparameter optimization (HPO) can be dfined as a blackbox optimization problem
  - Define $f(\boldsymbol{\lambda}) := \mathcal{L}_{HPO}(\mathcal{A}_{\boldsymbol{\lambda}}, \mathcal{D}_{train}, \mathcal{D}_{valid})$

Bayesian Optimization: Origin of the Name

- Bayesian optimization uses Bayes' theorem:

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)} \propto P(B|A) \times P(A)$$

- Bayesian optimization uses this to compute a posterior over functions:

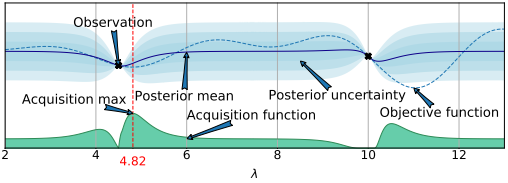
$$P(f|\mathcal{D}_{1:t}) \propto P(\mathcal{D}_{1:t}|f) \times P(f), \qquad \text{where } \mathcal{D}_{1:t} = \{\boldsymbol{\lambda}_{1:t}, c(\boldsymbol{\lambda}_{1:t})\}$$

- Meaning of the individual terms:
  - $P(f)$ is the prior over functions, which represents our belief about the space of possible objective functions before we see any data
  - $\mathcal{D}_{1:t}$ is the data (or observations, evidence)
  - $P(\mathcal{D}_{1:t}|f)$ is the likelihood of the data given a function
  - $P(f|\mathcal{D}_{1:t})$ is the posterior probability over functions given the data

Bayesian Optimization of a blackbox function in a nutshell

General approach

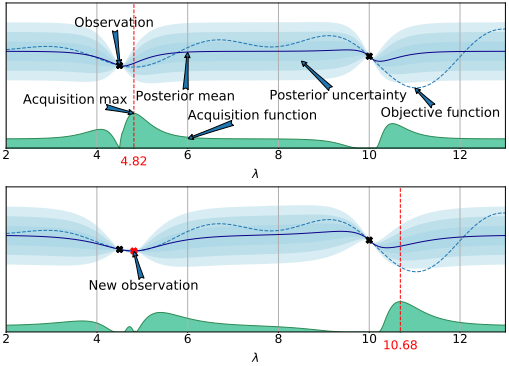
- Fit a probabilistic model to the collected function samples $\langle \boldsymbol{\lambda}, c(\boldsymbol{\lambda}) \rangle$
- Use the model to guide optimization, trading off exploration *vs* exploitation



Bayesian Optimization of a blackbox function in a nutshell

General approach

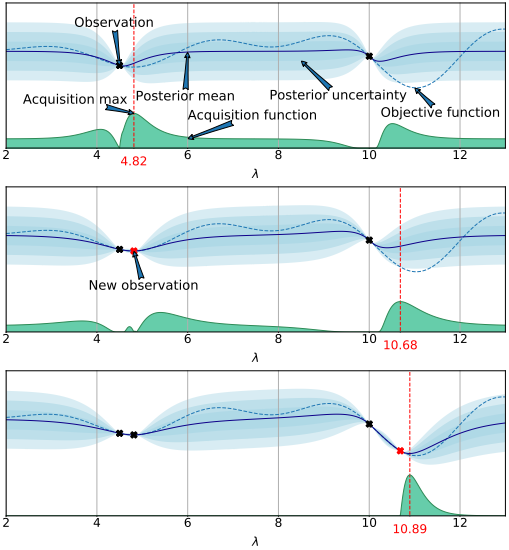
- Fit a probabilistic model to the collected function samples $\langle \boldsymbol{\lambda}, c(\boldsymbol{\lambda}) \rangle$
- Use the model to guide optimization, trading off exploration *vs* exploitation



Bayesian Optimization of a blackbox function in a nutshell

General approach

- Fit a probabilistic model to the collected function samples $\langle \boldsymbol{\lambda}, c(\boldsymbol{\lambda}) \rangle$
- Use the model to guide optimization, trading off exploration *vs* exploitation



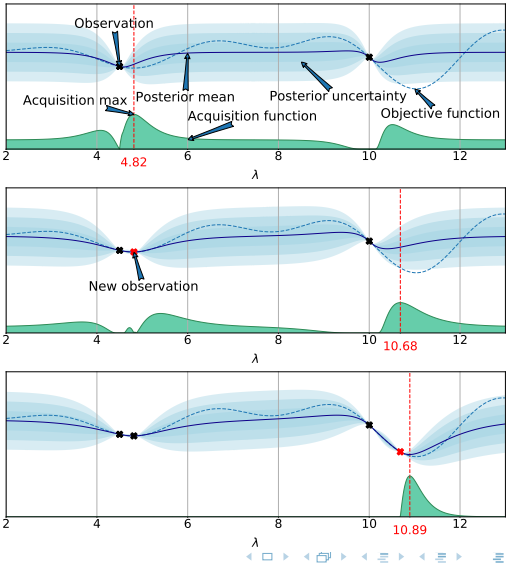
Bayesian Optimization of a blackbox function in a nutshell

General approach

- Fit a probabilistic model to the collected function samples $\langle \boldsymbol{\lambda}, c(\boldsymbol{\lambda}) \rangle$
- Use the model to guide optimization, trading off exploration *vs* exploitation

Popular approach in the statistics literature since [Mockus. 1975]

- Efficient in #function evaluations
- Works when objective is nonconvex, noisy, has unknown derivatives, etc.
- Recent convergence results [Srinivas et al. 2009; Bull. 2011; de Freitas et al. 2012; Kawaguchi et al. 2015]



Bayesian Optimization: Advantages and Disadvantages

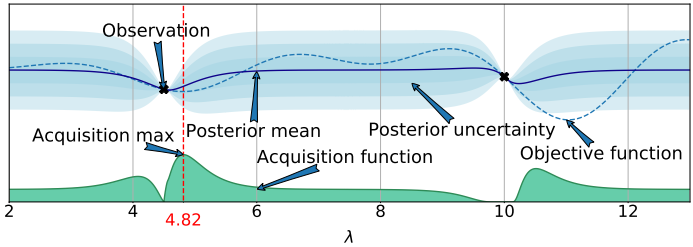
Advantages

- Sample efficient
- Can handle noise
- Native incorporation of priors
- Does not require gradients
- Theoretical guarantees

Disadvantages

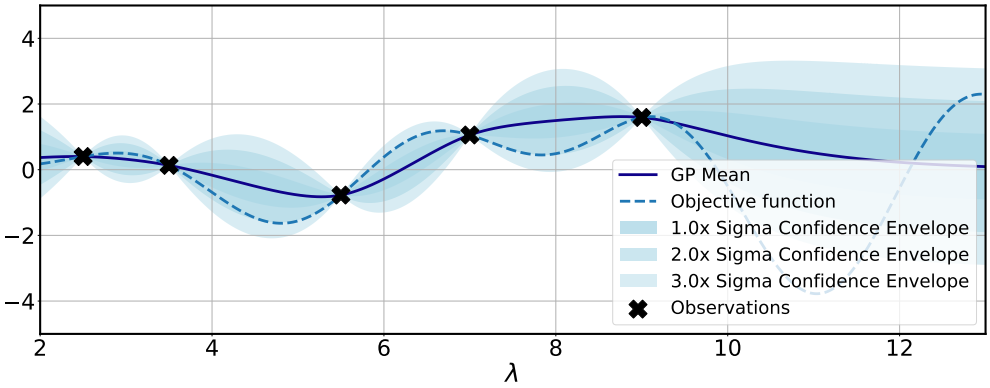
- Overhead because of model training in each iteration
- Crucially relies on robust surrogate model

Acquisition Functions: the Basics



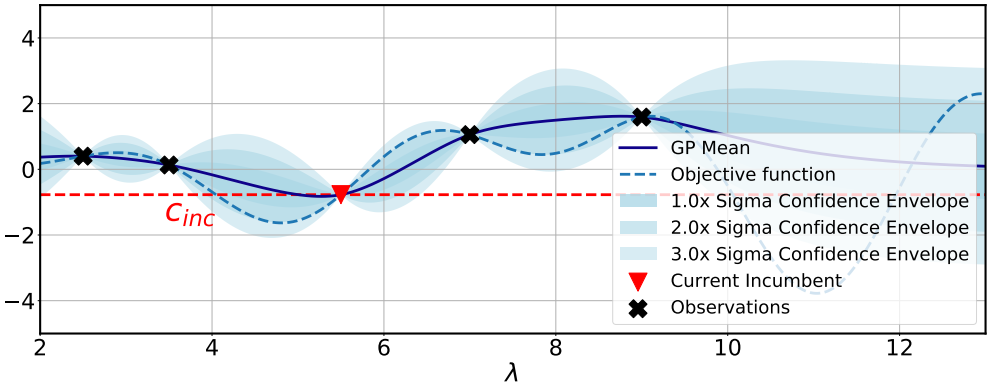
- Given the surrogate model $\hat{c}^{(t)}$ at the t-th iteration of BO, the acquisition function $u(\cdot)$ judges the utility (or usefulness) of evaluating f at $\boldsymbol{\lambda}^{(t)} \in \boldsymbol{\Lambda}$ next
- The acquisition function needs to trade off exploration and exploitation
  - E.g., just picking the $\boldsymbol{\lambda}$ with lowest predicted mean would be too greedy
  - We also need to take into account the uncertainty of the surrogate model $\hat{c}^{(t)}$ to explore

Probability of Improvement (PI): Concept



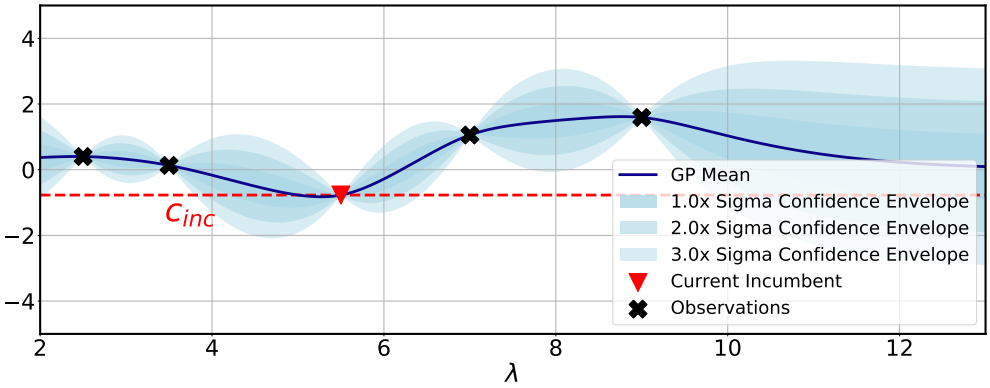
Given the surrogate fit at iteration t

Probability of Improvement (PI): Concept



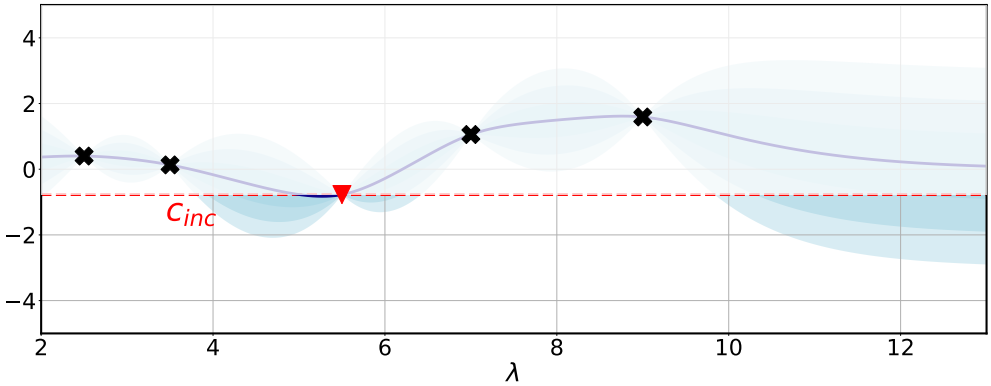
Current incumbent $\hat{\boldsymbol{\lambda}}$ and its observed cost c_{inc}

Probability of Improvement (PI): Concept



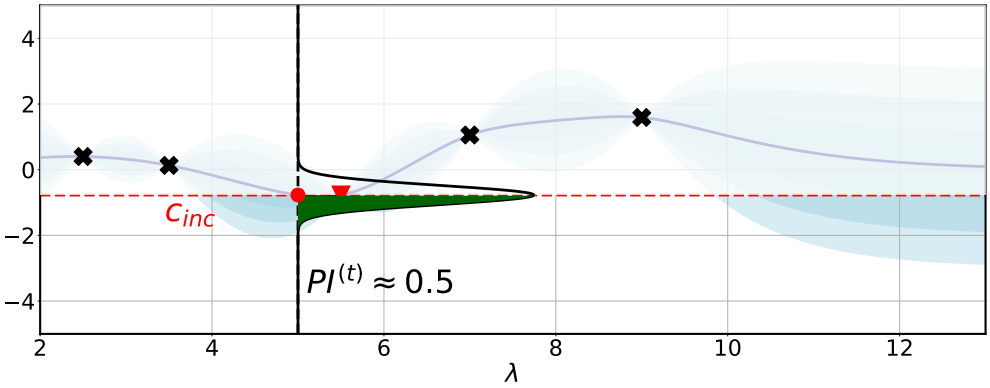
Now let's drop the objective function - it's unknown after all!

Probability of Improvement (PI): Concept



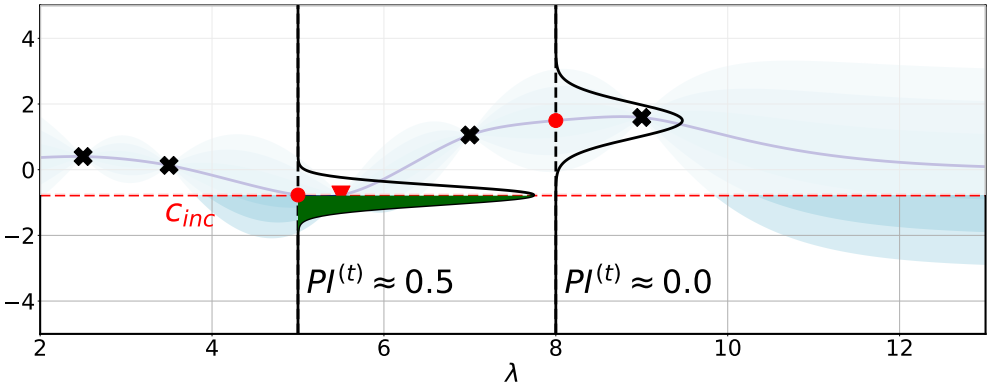
Intuitively, we care about the probability of improving over the current incumbent

Probability of Improvement (PI): Concept



PDF of a good candidate configuration. Only the green area is an improvement.

Probability of Improvement (PI): Concept



PDF of a bad candidate configuration

Probability of Improvement (PI): Formal Definition

- We define the current incumbent at time step t as: $\hat{\boldsymbol{\lambda}}^{(t-1)} \in \arg\min_{\boldsymbol{\lambda}' \in \mathcal{D}^{(t-1)}} c(\boldsymbol{\lambda}')$
- We write c_{inc} shorthand for the cost of the current incumbent: $c_{inc} = c(\hat{\boldsymbol{\lambda}}^{(t-1)})$
- The probability of improvement $u_{PI}(\boldsymbol{\lambda})$ at a configuration $\boldsymbol{\lambda}$ is then defined as:

$$u_{PI}^{(t)}(\boldsymbol{\lambda}) = P(c(\boldsymbol{\lambda}) \leq c_{inc}).$$

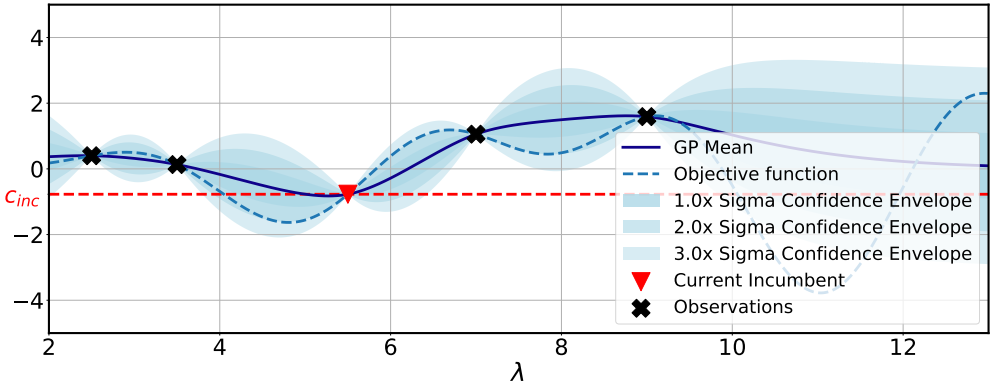
- Since the predictive distribution for $c(\boldsymbol{\lambda})$ is a Gaussian $\mathcal{N}(\mu^{(t-1)}(\boldsymbol{\lambda}), \sigma^{2(t-1)}(\boldsymbol{\lambda}))$, this can be written as:

$$u_{PI}^{(t)}(\boldsymbol{\lambda}) = \Phi[Z], \quad \text{with } Z = \frac{c_{inc} - \mu^{(t-1)}(\boldsymbol{\lambda}) - \xi}{\sigma^{(t-1)}(\boldsymbol{\lambda})},$$

where $\Phi(\cdot)$ is the CDF of the standard normal distribution and ξ is an optional exploration parameter

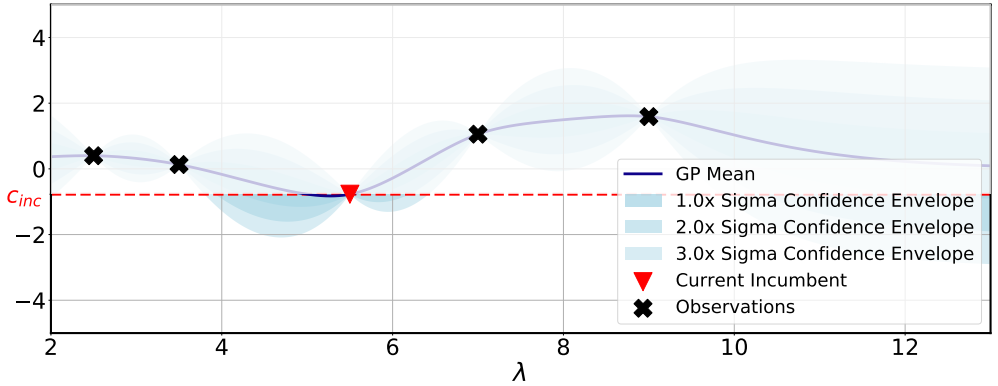
$$\boxed{\text{Choose } \boldsymbol{\lambda}^{(t)} \in \arg\max_{\boldsymbol{\lambda} \in \boldsymbol{\Lambda}} (u_{PI}^{(t)}(\boldsymbol{\lambda}))}$$

Expected Improvement (EI): Concept



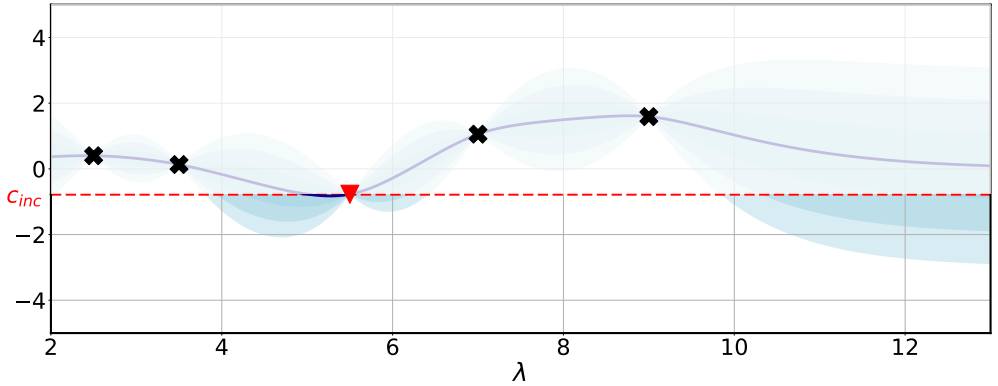
Given the surrogate fit at iteration t

Expected Improvement (EI): Concept



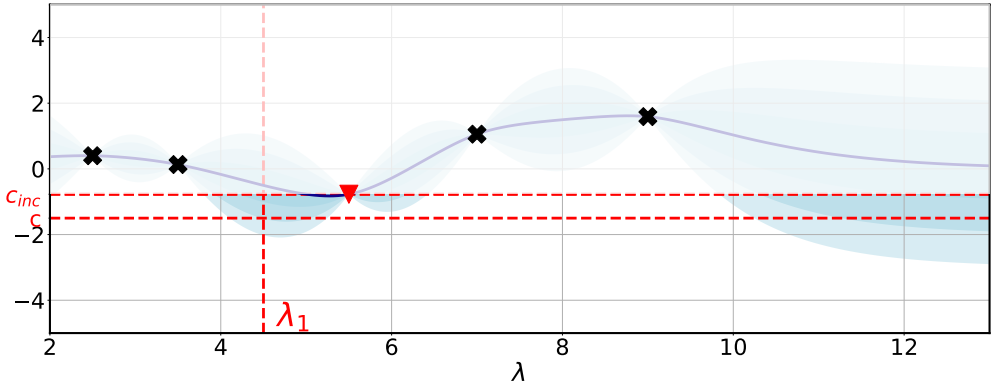
Region of probable improvement – but how large is the improvement?

Expected Improvement (EI): Concept



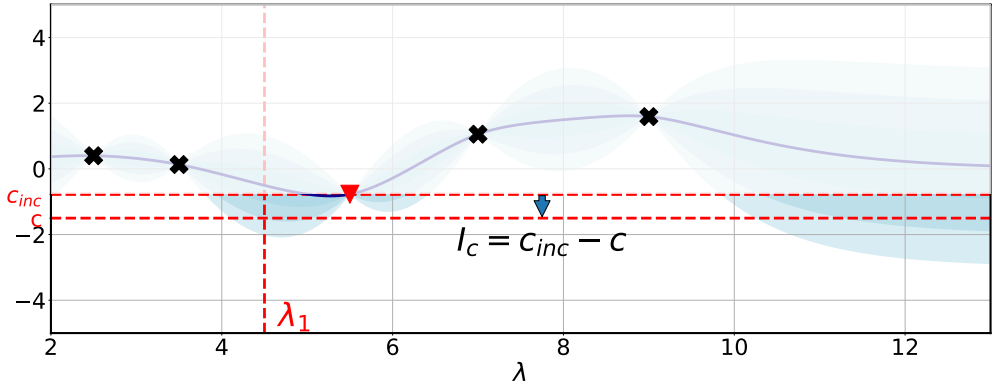
Region of probable improvement – but how large is the improvement?

Expected Improvement (EI): Concept



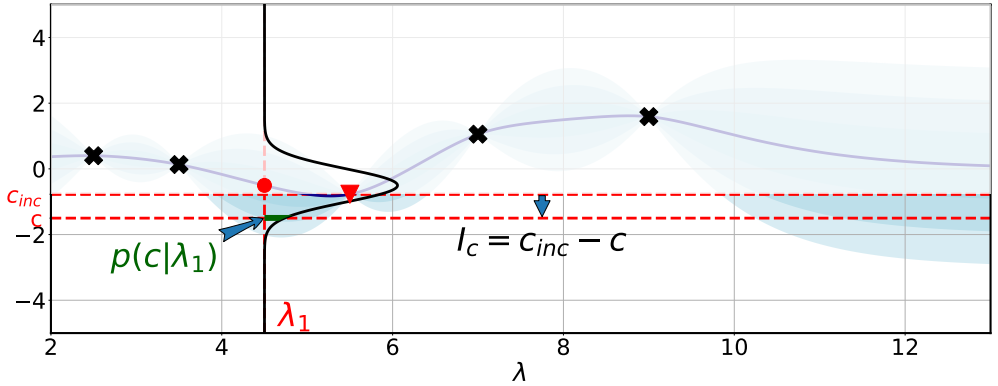
Hypothetical *real* cost c at a given $\boldsymbol{\lambda}$ - unknown in practice without evaluating

Expected Improvement (EI): Concept



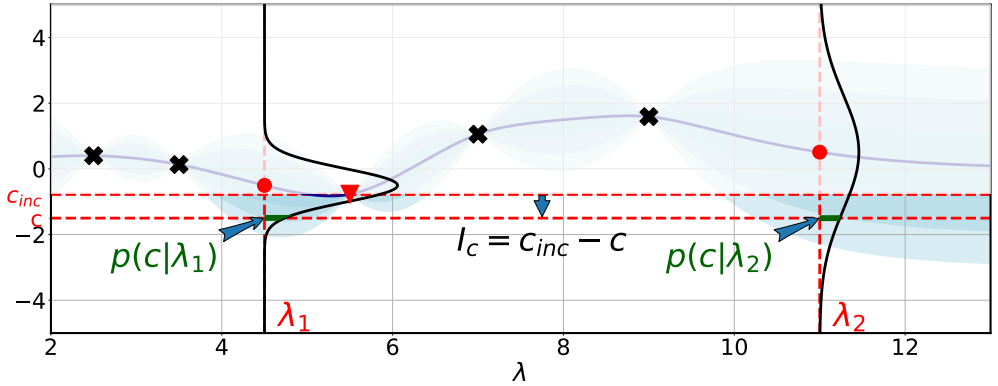
Given a hypothetical c, we can compute the improvement $I_c(\boldsymbol{\lambda})$

Expected Improvement (EI): Concept



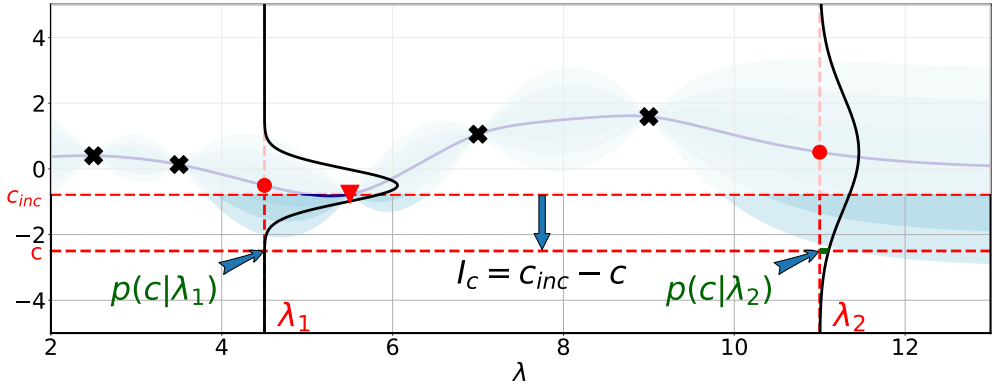
Given $\hat{c}(\boldsymbol{\lambda}) = \mathcal{N}(\mu(\boldsymbol{\lambda}), \sigma^2(\boldsymbol{\lambda}))$, we can also compute $p(c|\boldsymbol{\lambda})$.

Expected Improvement (EI): Concept



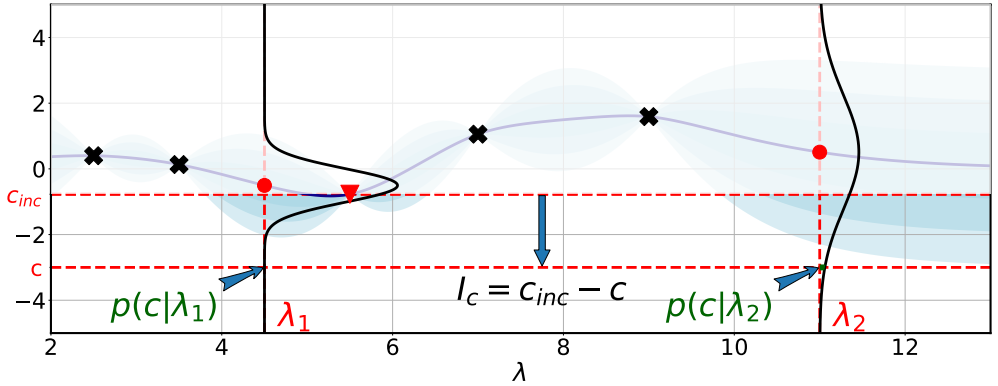
Compare the likelihood of a given improvement for two different configurations $\boldsymbol{\lambda}_1$ and $\boldsymbol{\lambda}_2$

Expected Improvement (EI): Concept



Now consider the likelihood of a larger improvement.

Expected Improvement (EI): Concept



Larger improvements are more likely in areas of high uncertainty.

To compute $\mathbb{E}[I(\boldsymbol{\lambda})]$, intuitively, we sum $p(c \mid \boldsymbol{\lambda}) \times I_c$ over all possibles values of c.

Expected Improvement (EI): Formal Definition

- We define the one-step positive improvement over the current incumbent as

$$I^{(t)}(\boldsymbol{\lambda}) = \max(0, c_{inc} - c(\boldsymbol{\lambda}))$$

- Expected Improvement is then defined as

$$u_{EI}^{(t)}(\boldsymbol{\lambda}) = \mathbb{E}[I^{(t)}(\boldsymbol{\lambda})] = \int_{-\infty}^{\infty} p^{(t)}(c \mid \boldsymbol{\lambda}) \times I^{(t)}(\boldsymbol{\lambda}) \; dc.$$

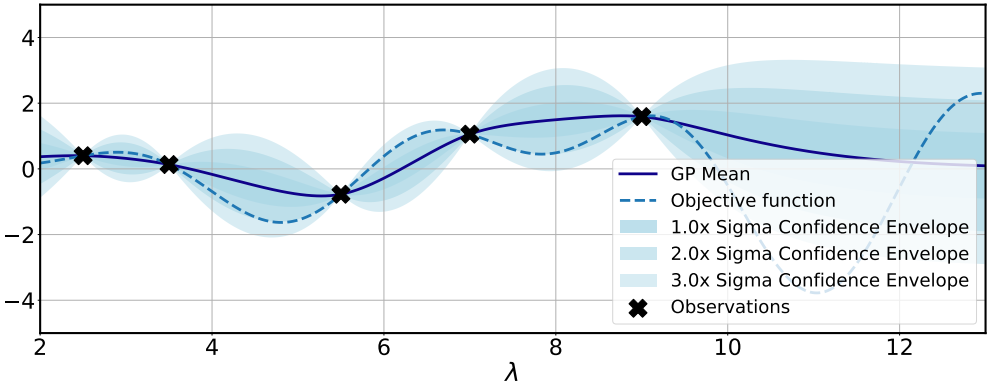
- Since the posterior distribution of $\hat{c}(\boldsymbol{\lambda})$ is a Gaussian, EI can be computed in closed form:

$$u_{EI}^{(t)}(\boldsymbol{\lambda}) = \begin{cases} \sigma^{(t)}(\boldsymbol{\lambda})[Z\Phi(Z) + \phi(Z)], & \text{if } \sigma^{(t)}(\boldsymbol{\lambda}) > 0 \\ 0 & \text{if } \sigma^{(t)}(\boldsymbol{\lambda}) = 0, \end{cases}$$

$$\text{where } Z = \frac{c_{inc} - \mu^{(t)}(\boldsymbol{\lambda})}{\sigma^{(t)}(\boldsymbol{\lambda})}$$

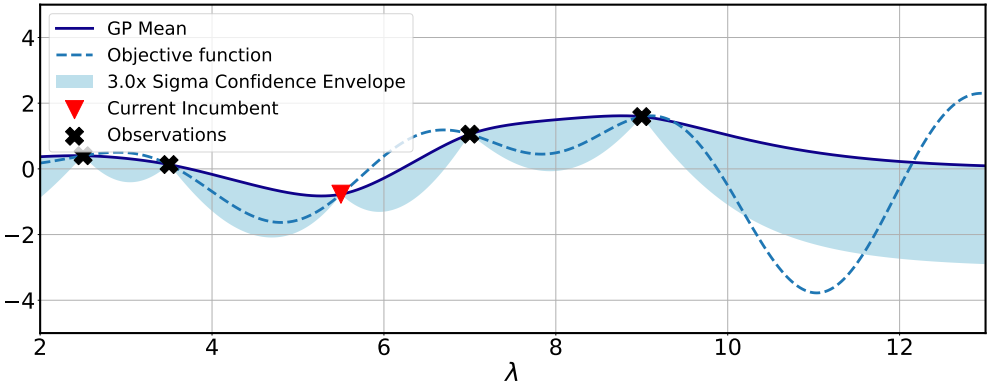
$$\boxed{\text{Choose } \boldsymbol{\lambda}^{(t)} \in \arg\max_{\boldsymbol{\lambda} \in \boldsymbol{\Lambda}} (u_{EI}^{(t)}(\boldsymbol{\lambda}))}$$

Lower/Upper Confidence Bounds (LCB/UCB): Concept



Given the surrogate fit at iteration t

Lower/Upper Confidence Bounds (LCB/UCB): Concept



Lower Confidence Bound, $\mu(\boldsymbol{\lambda}) - \alpha\sigma(\boldsymbol{\lambda})$ (here, for $\alpha = 3$)

Lower/Upper Confidence Bounds (LCB/UCB): Formal Definition

- We define the Lower Confidence Bound as

$$u_{LCB}^{(t)}(\boldsymbol{\lambda}) = \mu^{(t)}(\boldsymbol{\lambda}) - \alpha\sigma^{(t)}(\boldsymbol{\lambda}), \quad \alpha \geq 0$$

- One can schedule α (e.g., increase it over time [Srinivas et al. 2009])

$$\boxed{\text{Choose } \boldsymbol{\lambda}^{(t)} \in \arg\max_{\boldsymbol{\lambda} \in \boldsymbol{\Lambda}} \left(-u_{LCB}^{(t)}(\boldsymbol{\lambda})\right)}$$

Lower/Upper Confidence Bounds (LCB/UCB): Formal Definition

- We define the Lower Confidence Bound as

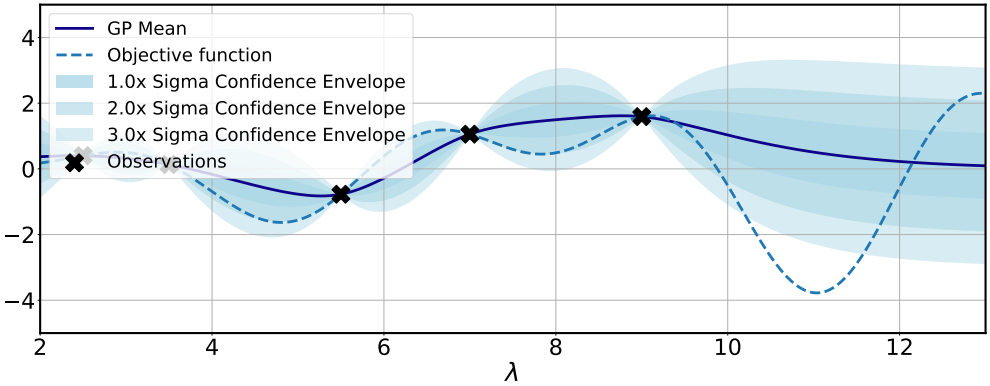
$$u_{LCB}^{(t)}(\boldsymbol{\lambda}) = \mu^{(t)}(\boldsymbol{\lambda}) - \alpha\sigma^{(t)}(\boldsymbol{\lambda}), \quad \alpha \geq 0$$

- One can schedule α (e.g., increase it over time [Srinivas et al. 2009])

$$\boxed{\text{Choose } \boldsymbol{\lambda}^{(t)} \in \arg\max_{\boldsymbol{\lambda} \in \boldsymbol{\Lambda}} \left(-u_{LCB}^{(t)}(\boldsymbol{\lambda})\right)}$$

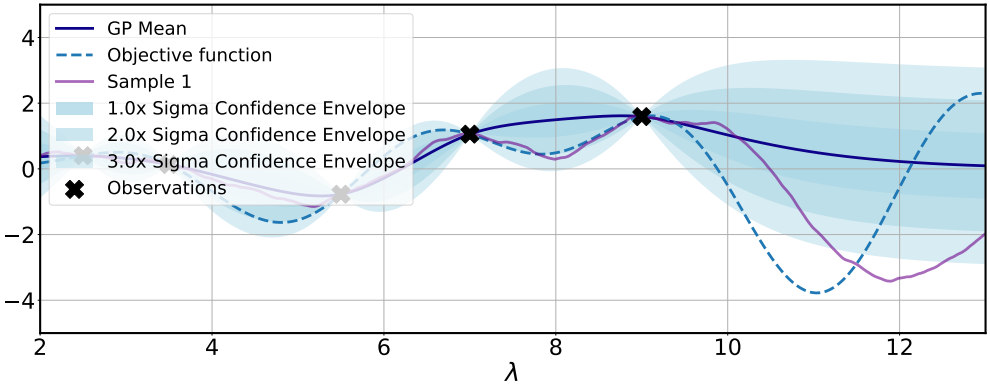
- Note: when one aims to maximize the objective function, one would use UCB instead
  - $u_{UCB}^{(t)}(\boldsymbol{\lambda})) = \mu^{(t)}(\boldsymbol{\lambda}) + \alpha\sigma^{(t)}(\boldsymbol{\lambda})$
  - For UCB, one would choose $\boldsymbol{\lambda}^{(t)} \in \arg\max_{\boldsymbol{\lambda} \in \boldsymbol{\Lambda}}(u_{UCB}^{(t)}(\boldsymbol{\lambda}))$

Thompson Sampling (TS): Concept



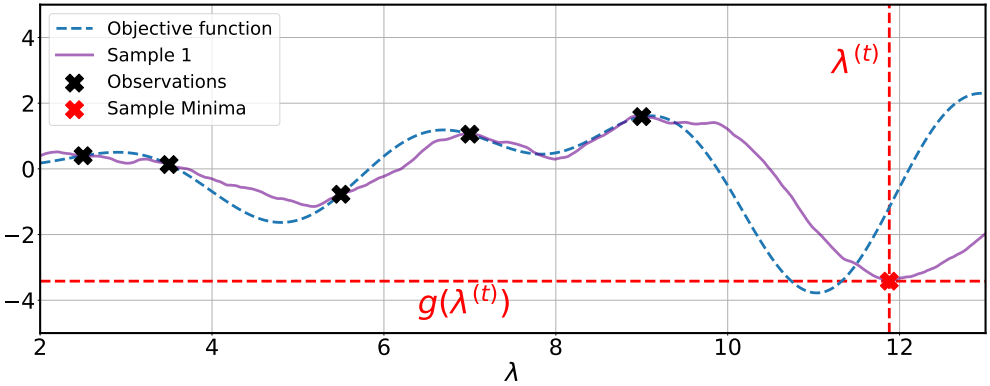
Given the surrogate at iteration t fit on dataset $\mathcal{D}^{(t-1)}$

Thompson Sampling (TS): Concept



Draw a sample g from the predictive surrogate model

Thompson Sampling (TS): Concept



Then choose the minimum of this sample to evaluate at next

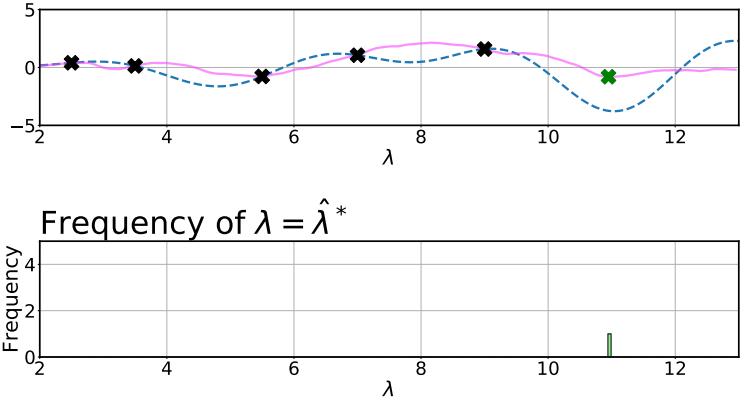
Entropy Search Preliminaries

- Key idea: Evaluate $\boldsymbol{\lambda}$ which most reduces our uncertainty about the location of $\boldsymbol{\lambda}^*$
- We'll use the p_{min} distribution to characterize the location of $\boldsymbol{\lambda}^*$:

$$p_{min}(\boldsymbol{\lambda}|\mathcal{D}) = p(\boldsymbol{\lambda} \in \arg\min_{\boldsymbol{\lambda}' \in \boldsymbol{\Lambda}}(\hat{c}(\boldsymbol{\lambda}')|\mathcal{D}))$$

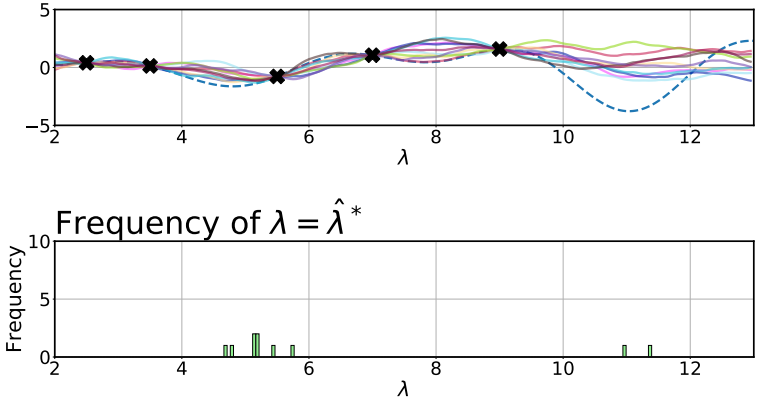
- Our uncertainty is then captured by the entropy $H(p_{min}(\cdot|\mathcal{D}))$ of the p_{min} distribution
- Minimizing $H(p_{min}(\cdot|\mathcal{D}))$ yields a peaked p_{min} distribution, i.e., strong knowledge about the location of $\boldsymbol{\lambda}^*$

Entropy Search: Visualization of the p_{min} Distribution



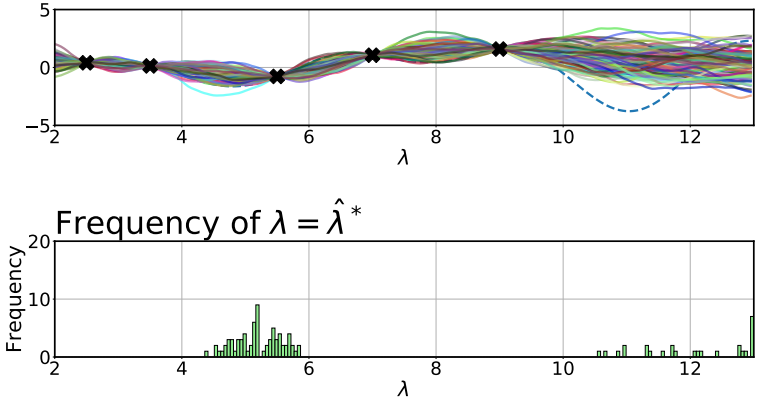
For each sample drawn from $\hat{c}$, we can compute where $\boldsymbol{\lambda}^*$ lies

Entropy Search: Visualization of the p_{min} Distribution



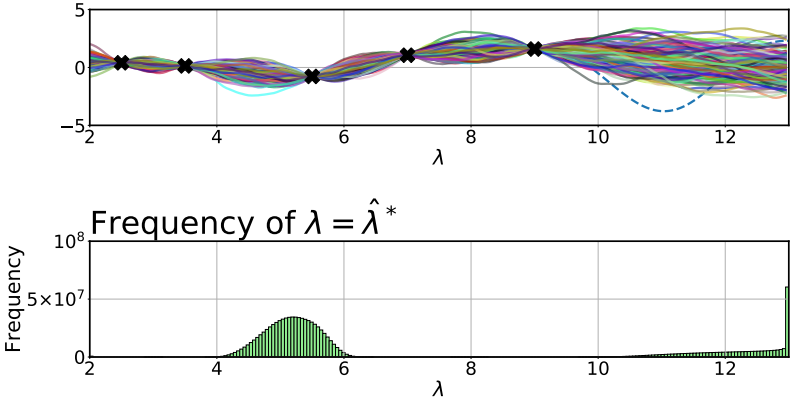
For each sample drawn from $\hat{c}$, we can compute where $\boldsymbol{\lambda}^*$ lies

Entropy Search: Visualization of the p_{min} Distribution



From many samples we can approximate the p_{min} distribution

Entropy Search: Visualization of the p_{min} Distribution



From many samples we can approximate the p_{min} distribution

Entropy Search: Formal Definition

- The p_{min} distribution characterizes the location of $\boldsymbol{\lambda}^*$:

$$p_{min}(\boldsymbol{\lambda}^*|\mathcal{D}) = p(\boldsymbol{\lambda}^* \in \arg\min_{\boldsymbol{\lambda}' \in \boldsymbol{\Lambda}}(\hat{c}(\boldsymbol{\lambda}')|\mathcal{D}))$$

Entropy Search: Formal Definition

- The p_{min} distribution characterizes the location of $\boldsymbol{\lambda}^*$:

$$p_{min}(\boldsymbol{\lambda}^*|\mathcal{D}) = p(\boldsymbol{\lambda}^* \in \arg\min_{\boldsymbol{\lambda}' \in \boldsymbol{\Lambda}}(\hat{c}(\boldsymbol{\lambda}')|\mathcal{D}))$$

- Our uncertainty about the location of $\boldsymbol{\lambda}^*$ is captured by the entropy $H(p_{min}(\cdot|\mathcal{D}))$ of the p_{min} distribution

Entropy Search: Formal Definition

- The p_{min} distribution characterizes the location of $\boldsymbol{\lambda}^*$:

$$p_{min}(\boldsymbol{\lambda}^*|\mathcal{D}) = p(\boldsymbol{\lambda}^* \in \arg\min_{\boldsymbol{\lambda}' \in \boldsymbol{\Lambda}}(\hat{c}(\boldsymbol{\lambda}')|\mathcal{D}))$$

- Our uncertainty about the location of $\boldsymbol{\lambda}^*$ is captured by the entropy $H(p_{min}(\cdot|\mathcal{D}))$ of the p_{min} distribution

- Entropy search aims to minimize $H(p_{min})$, to yield a peaked p_{min} distribution:

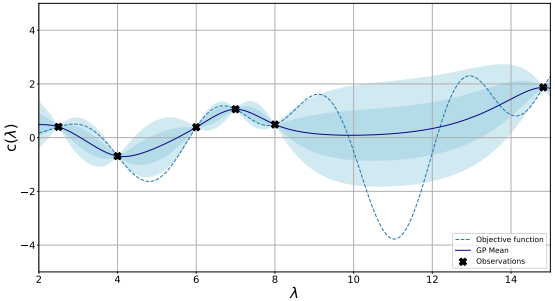
$$u_{ES}(\boldsymbol{\lambda}) = H(p_{min}(\cdot|\mathcal{D})) - \mathop{\mathbb{E}}_{\tilde{c} \sim \hat{c}(\lambda)^{(t)}} H(p_{min}(\cdot|\mathcal{D} \cup \{\langle \boldsymbol{\lambda}, \tilde{c} \rangle\}))$$

$$\text{Choose } \boldsymbol{\lambda}^{(t)} = \arg\max_{\boldsymbol{\lambda} \in \boldsymbol{\Lambda}}(u_{ES}^{(t)}(\boldsymbol{\lambda}))$$

Desiderata for Surrogate Models in Bayesian Optimization

In all cases

- Regression model with uncertainty estimates
- Accurate predictions



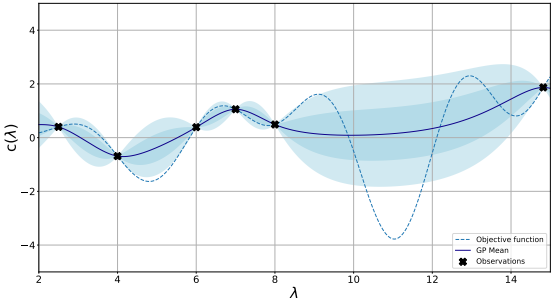
Desiderata for Surrogate Models in Bayesian Optimization

In all cases

- Regression model with uncertainty estimates
- Accurate predictions

Examples:

- Gaussian Processes
- Bayesian Neural Networks
- Random Forests



Gaussian Processes (GPs): Reminder of Pros and Cons

Advantages

- Smooth and reliable uncertainty estimates
- Strong sample efficiency
- We can encode expert knowledge about the design space in the kernel

These advantages make GPs the most commonly-used model in Bayesian optimization

Disadvantages

- Performance can be quite sensitive to the choice of kernel
- Cost scales cubically with the number of observations
- Weak performance for high dimensionality
- Not easily applicable in discrete or conditional spaces
- Sensitive to its own hyperparameters

Bayesian Neural Networks (BNNs): Overview of Pros and Cons

Advantages

- Scales linearly with #observations
- Can obtain nice and smooth uncertainty estimates
- Flexibility: handling of categorical, continuous and discrete spaces

Disadvantages

- Usually needs more data than Gaussian processes
- Uncertainty estimates often worse than for Gaussian processes
- Many meta-design decisions
- No robust off-the-shelf implementation

These qualities make BNNs an ever-more promising alternative

Beyond the Standard Bayesian Optimization Setting

Standard Bayesian optimization problems

- Low-dimensional functions
- Continuous, smooth functions
- Sequential optimization

Extensions

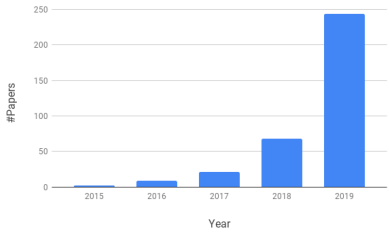
- Structured search spaces: categorical & conditional hyperparameters
- High dimensions
- Parallel evaluations
- Optimization with constraints

Neural Architecture Search (NAS)

- Goal: automatically find neural architectures with strong performance
  - Optionally, subject to a resource constraint

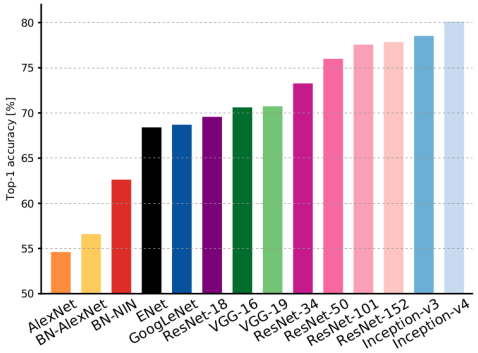
- A decade-old problem, but main stream since 2017 and now intensely researched
- One of the main problems AutoML is known for

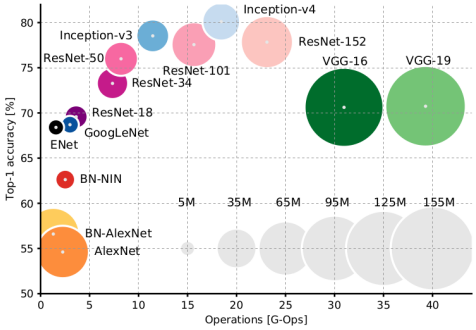
- Initially extremely expensive
- By now several methods promise low overhead over a single model training



Motivation for NAS

- Performance improvements on various tasks due to novel architectures
- Can we automate this design process, potentially discovering new components/topologies?

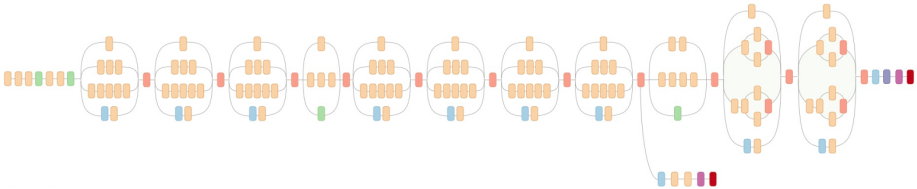




[Canziani et al. 2017]

Motivation for NAS

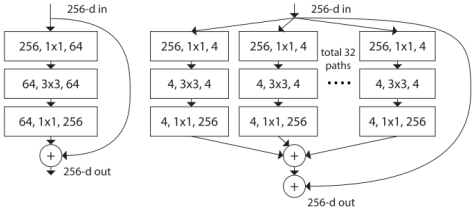
- Manual design of architectures is time consuming
- Complex state-of-the-art architectures are a result of years of trial and errors by experts



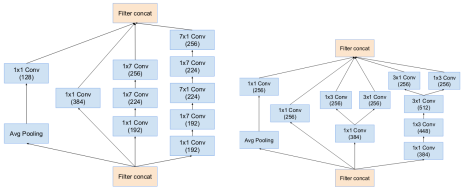
Inception-v3 [Szegedy et al. 2015]

Motivation for NAS

- Manual design of architectures is time consuming
- Complex state-of-the-art architectures are a result of years of trial and errors by experts
  - Main pattern: Repeated blocks with same structure (topology)

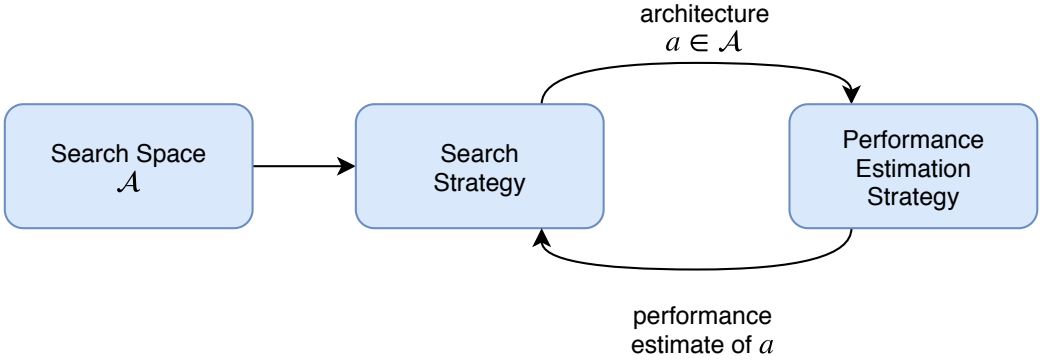


ResNet/ResNeXt blocks
[He et al. 2016; Xie et al. 2016]



Inception-v4 blocks [Szegedy et al. 2016]

NAS components [Elsken et al. 2019]



- Search Space: the types of architectures we consider; micro, macro, hierarchical, etc.
- Search Strategy: Reinforcement learning, evolutionary strategies, Bayesian optimization, gradient-based, etc.
- Performance Estimation Strategy: validation performance, lower fidelity estimates, one-shot model performance, etc.

Bibliography:

- This lecture is based on:
  - **AutoML course**, Marius Lindauer, Frank Hutter, Bernd Bischl, Lars Kotthoff, Janek Thomas `https://ki-campus.org/index.php/courses/automl-luh2021`
- Bibliography:
  - Hutter, Frank; Kotthoff, Lars; Vanschoren, Joaquin Automated Machine Learning - Methods, Systems, Challenges Springer, 2019.
    `https://link.springer.com/chapter/10.1007/978-3-030-05318-5_1`
  - Roman Garnett, Bayesian Optimization, Cambridge University Press, 2023
    `https://bayesoptbook.com/`
- License:
  - Creative Commons Attribution-ShareAlike 4.0 International License.
    `https://creativecommons.org/licenses/by-sa/4.0/`